Aircraft-Like Voxel-Geometry Generation with a CFD-Informed Evaluation Path: Measured Validity on a Declared Held-Out Split

Darsh Gupta, National Public School, HSR Layout, Bengaluru
darshgu77@gmail.com

Abstract—This paper targets aircraft-like voxel-geometry generation: the repository combines a latent diffusion-style model, voxel decoders, an internal D3Q27 lattice-Boltzmann scoring path, and an OpenFOAM export route, and the goal is generated voxel geometries that pass the repository's own aircraft-specific validity checks. We report a measured repair of that capability. Five root causes of the earlier total validity failure are isolated by measurement: a procedural training corpus whose private gate admitted shapes the declared screen rejects (1 of 47 admitted samples passed), a decoder that emitted a near-constant probability field, decoder and consistency-student learning rates that never received the requested value, generation that sampled only an under-trained consistency student, and latent-capacity/corpus-scale limits. Under a declared protocol — six NASA CRM records for training, nine held out (six distinct voxel arrays), an uncalibrated 0.5 materialization threshold, and the unmodified validity screen — the retrained 32^3 checkpoint reconstructs 9/9 held-out records and generates 45/45 samples that pass every aircraft-specific check, re-screened by the repository's own CLI. The result is bounded and is reported as such: the 45 draws contain five near-identical voxel arrays, every emitted voxel is present in the training corpus, and a model-free constant array passes the same screen; the current heuristic gate is therefore satisfiable by a training-hull emitter, while novel, conditioned, and diverse generation are not demonstrated. The paper's measurement-integrity findings remain part of the evidence: a reference-area defect changed a formerly passing comparison row from 0.25% to 14.2%, the corrected frozen corpus meets the 5% gate on 1/16 sonicFoam and 0/14 simpleFoam rows, and the condition-response study is non-significant. Aerodynamic optimization, structural viability, manufacturing validity, and physical-performance claims remain blocked.

Index Terms—Generative Design, Diffusion Models, Computational Fluid Dynamics, Aerospace Engineering, Voxel Geometry.

I. Introduction

GENERATIVE design is increasingly used to explore large engineering design spaces, while aerospace workflows continue to rely heavily on computational fluid dynamics (CFD), surrogate models, and optimization loops to evaluate aerodynamic tradeoffs [1], [2]. Aircraft design adds further constraints beyond raw shape generation, including manufacturability, structural load paths, and mission-specific performance targets. Those requirements make it easy to overstate what a proof-of-concept generative repository can presently demonstrate.

This paper targets that concrete objective: generated aircraft-like voxel geometries that pass the repository's own aircraft-specific validity checks. The repository implements a latent diffusion-style pipeline that maps latent samples to voxel geometries, scores them with an internal CFD-oriented path, and exports them for downstream validation. We report the measurement-driven repair of the generation path, the declared held-out protocol under which the repaired checkpoint is evaluated, and the parts of the objective that remain unmet. The CFD-oriented evaluation path and the measurement-integrity findings that keep it honest are reported as supporting evidence, because a generation result that is not separated from its evaluation path cannot be interpreted.

The paper makes four bounded contributions:

1) A measured diagnosis of five root causes behind the total aircraft-validity failure of the earlier checkpoints, each isolated on a single 8 GiB host with a recorded command: corpus/gate mismatch, decoder collapse, unreachable decoder learning rates, an under-trained sampling path, and latent-capacity limits.
2) An executed repair under a declared protocol: nine held-out NASA CRM records (six distinct voxel arrays), an uncalibrated materialization threshold, and the unmodified screen, giving 9/9 reconstruction and 45/45 generated samples that pass, re-screened by the repository's own CLI — together with the measured caveat that every emitted voxel exists in the training corpus and a model-free constant array passes the same screen.
3) A test-first correction to a reference-area mismatch in the LBM/OpenFOAM comparison path, with regression tests and corrected frozen-corpus results, plus a reproducibility-forensics record of solver instability and the limits of an n=12 condition-response design.
4) A claim-gating protocol that separates what is measured from what is fixture-only, and that records which gates are met in their literal form and which remain blocked by missing baselines, external solvers, structural analysis, or higher-compute runs.

The remainder of the paper first places the repository beside 3D generative modeling, topology optimization, and CFD-driven design practice, then describes the implementation that exists in the codebase. It then reports

the reduced final-protocol evidence, records the validation requirements that still block stronger claims, and closes by summarizing the present scope and the minimum future work needed for a conditioned airplane generator.

II. Related Work

Diffusion models are now a standard point of reference for high-quality generative modeling [3], [4]. In 3D generation, recent systems such as Point-E and Shap-E show that diffusion-style methods can generate point-cloud or implicit 3D assets at practical sampling speeds [5], [6]. Those works are important context because they establish that diffusion models can represent 3D geometry, but they do not by themselves solve aircraft-specific design, manufacturability, or validation concerns.

An adjacent engineering-design tradition comes from topology optimization, where the goal is to optimize material layout or geometry under physics-based objectives and constraints. That literature is mature and has a well-developed vocabulary for separating optimization variables, constraints, and validation requirements [7]. Topology Optimization (TO), including the SIMP method [8], remains a central baseline for load-path optimization [9]. Recent neural TO approaches [10] accelerate parts of this process. The present repository is closer to a generative prior over voxelized shapes than to classical topology optimization, but TO practice remains a useful reference for how strongly structural or performance claims should be validated.

Conventional aerodynamic shape optimization couples CFD solvers with numerical optimization over parameterized geometries, often using gradient or adjoint information [1], [11], [12]. Surrogate modeling is also widely used for efficient design-space exploration [2], [13]. In those settings, the geometry representation, optimizer, and solver are explicit components of a controlled loop. By contrast, the present repository samples from a learned latent model and then applies CFD-style evaluation. This makes the workflow exploratory, and success in code-path execution should not be confused with the stronger evidence expected in mature CFD-based shape optimization.

Physics-informed machine learning incorporates PDEs through PINNs [14], Neural Operators such as FNO [15], and operator-learning models such as DeepONet [16]. Differentiable-simulation work goes further by embedding differentiable fluid solvers into learning and design frameworks [17]. These directions represent stronger forms of "simulation in the loop" than the reduced sanity evidence reported here. The current repository uses an internal LBM-style scoring path as an implementation mechanism; using that path to promote labels across fidelity levels after validation remains future work [18].

Constraint-aware generation is another relevant line of work. Enforcing physical validity, including manufacturability and connectivity, remains a core challenge [19]. Oh et al. [20] demonstrated generative design for structures, while Steinmetz et al. [21] used differentiable projection for manufacturable designs. The current repository evidence is more limited: connectivity penalties, post-processing cleanup, and manufacturing-aware scoring proxies are useful for smoke-testing the code path, but they are not aircraft-specific validity guarantees.

The implementation also borrows standard architectural ingredients from modern deep learning, including attention-style modules inspired by the Transformer literature [22]. In this paper, those ingredients are treated as reused components rather than as a novelty claim.

A. Positioning and Novelty

The present repository does not claim a new diffusion objective, a new adjoint CFD method, or a validated aircraft-design benchmark. Its contribution is a measured, bounded generation result together with the measurement discipline that makes it interpretable: aircraft-valid voxel geometry generated from a checkpoint trained and evaluated on an 8 GiB host under a declared held-out split and an uncalibrated screen, paired with an explicit claim ledger, a test-first reference-area correction, instability and same-seed reproducibility forensics, and fail-closed gates. Relative to general 3D diffusion work, this paper studies what must be measured before engineering-shaped voxel outputs can support design claims — and shows that passing a shape-validity gate is necessary but not sufficient for generative diversity. Relative to topology optimization and classical CFD shape optimization, the repository is a lower-fidelity generative prior whose validity evidence is now positive at 32^3 on a single aircraft family while its aerodynamic evidence remains bounded. Relative to surrogate and differentiable-simulation papers, the present work contributes audit evidence rather than a validated replacement or performance result.

This positioning is intentionally conservative. The repository should not yet be read as demonstrating superior exploration of an engineering design space, validated surrogate replacement, conditioned aircraft generation, or manufacturing-conditioned aircraft generation. Those claims require the stronger evidence gates summarized below and in Section V.

B. Baseline and Ablation Protocol

The reduced final evidence package requires three executable baseline families: retrieval records from the grounded manifest, unconditional samples from the checkpoint, and bundled grounded STL examples. These are guardrail baselines: they verify that the report contains concrete comparison artifacts and repeated-run statistics, but they do not establish superiority over classical optimization or mature generative-model baselines.

Publication-grade baseline comparisons should still include four stronger families:

1) Classical primitives: hand-designed parametric aircraft primitives, such as swept or tapered wings extruded from NACA airfoil profiles, with controlled aspect-ratio, sweep, and taper ranges.

TABLE I
Experimental Plan and Claim Mapping Beyond the Current Guardrail Baselines

Claim gated	Baseline or ablation	Metric family	Execution requirement
Aerodynamic efficiency	Classical primitives; adjoint ASO; no-CFD-loss ablation	C_L/C_D; drag coefficient; solver agreement	Shared geometry normalization, consistent reference area, and a documented CFD-resolution sweep.
Manifold integrity	Voxel-GAN; point-cloud diffusion; no-cleanup ablation	Connected-component ratio; repair success rate	Voxel-connectivity analysis on a fixed generated sample set and aircraft-specific geometric checks.
Computational efficiency	Random search with CFD ranking; long-step diffusion baseline	Inference latency; memory use; score stability	Wall-clock timing on declared hardware with fixed batch size and repeated runs.
Generative diversity	Parametric primitive library and conditioned batches	Shape similarity; latent coverage; part-volume variance	Grounded aircraft-like corpus, declared condition schema, and held-out condition-response tests.

2) Unconstrained generative 3D models: representative voxel, point-cloud, and implicit-shape baselines such as voxel-GAN [23], point-cloud diffusion [24], and occupancy networks [25].
3) Independent optimization search: a SIMP-style topology-optimization comparison [8] and a random-search baseline in which sampled candidates are ranked by the same CFD evaluation protocol.
4) Differentiable aerodynamic shape optimization: an adjoint-based refinement baseline for a standard fuselage-wing primitive [12].

Ablations should isolate the impact of aerodynamic scoring diagnostics, connectivity diagnostics, the connected-component cleanup hook, the semantic constraint projector, and the consistency-step count. For now, that list is a protocol rather than a causal result; it becomes evidence only after larger grounded aircraft-like runs with shared solver settings.

III. Methodology

The current repository implements a proof-of-concept generative workflow built from four components: a noise scheduler, a latent diffusion-style denoiser, a latent-to-3D converter, and an internal CFD-oriented evaluator. The codebase now has two evidence tracks: deterministic synthetic/procedural smoke data retained for continuity, and a public VSP Airshow corpus used for the grounded training smoke run reported in Section IV. In this revision, the same path also carries a documented structured condition vector through dataset generation, latent construction, and checkpointed inference. Figure 1 should therefore be read as an implementation diagram for the present repository rather than as evidence of a fully validated aircraft-design system.

A. Public Airshow Corpus Construction

The grounded corpus path collects public model documents from the VSP Airshow web app, keeps only records whose Airshow license identifiers map to CC0, CC BY, or CC BY-SA, and requires a public preview-geometry URL before a record can enter the training manifest [26]–[28]. The builder parses available X3D indexed-face-set geometry, normalizes each mesh into a centered unit cube, voxelizes the occupied geometry into a fixed grid, and records source URLs, license metadata, geometry hashes, voxel hashes, split assignment, and preprocessing provenance. The primary smoke run uses a 16^3 grid, and the later resolution addendum reruns the same corpus construction at 32^3, 64^3, and 96^3. The deterministic split assignment is stored in the manifest so that later validation, training, and generation commands can be tied back to the same corpus lineage.

The run used in this paper observed 381 public Airshow model documents, admitted 357 license-and-geometry-eligible documents, and converted 355 records after two public storage URLs returned 404. Figure 2 visualizes that corpus funnel together with the license and split distributions. Airshow model names, manufacturer fields, URLs, and license labels are treated as source metadata only. The additional mission and manufacturing fields used by the condition vector are deterministic repository inferences from geometry and defaults, not factual claims made by NASA, Lockheed, OpenVSP, or any other named source.

B. Noise Scheduling

The forward diffusion process gradually adds Gaussian noise to the input data over a series of T timesteps. The noise schedule is defined by a variance schedule β_t, which is typically a linear or cosine schedule. The implementation uses a linear schedule, where β_t increases linearly from β_{start} to β_{end}.

The forward process is defined as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{1}$$

During reverse sampling, the denoiser estimates the noise component of a noisy latent. The training loss compares predicted noise with sampled noise; sampling then uses that estimate to update the latent.

C. Latent Diffusion UNet

The denoising model is a UNet-based architecture that operates in the latent space. The UNet takes as input a noisy latent vector, a timestep embedding, and an optional condition embedding, and outputs the predicted noise. The implementation projects the latent to a small spatial

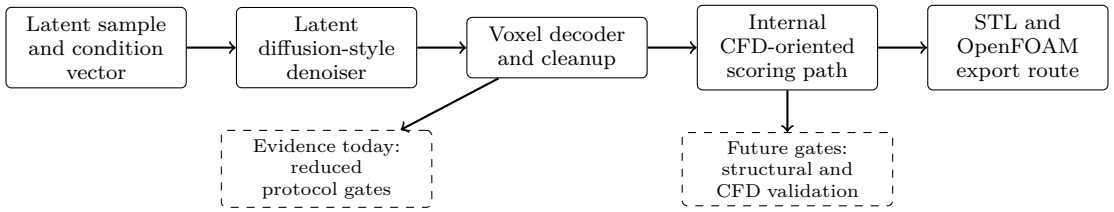


Fig. 1. Implementation-level view of the current repository. A latent code and optional structured condition vector are routed through denoising, voxel decoding, cleanup, internal scoring, and export paths. The diagram describes the code path under test; it does not by itself establish differentiable training, structural viability, or publication-grade CFD validation.

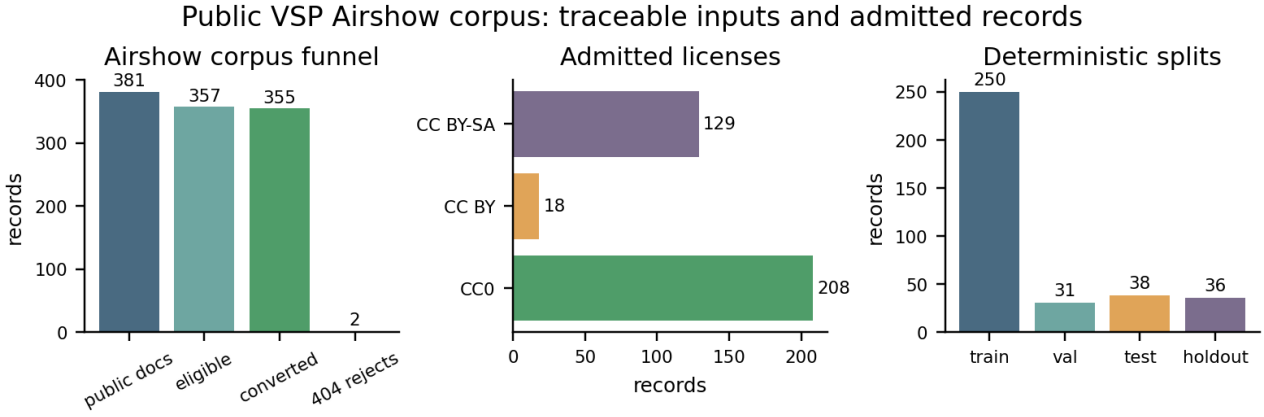


Fig. 2. Corpus construction summary for the public VSP Airshow smoke run. The plot shows the observed Airshow model-document count, license-and-geometry filtering, converted voxel records, admitted license mix, and deterministic train/validation/test/holdout split. These counts document data provenance and coverage; they do not certify manufacturer approval or aircraft performance.

tensor and applies residual block stacks with additive skip connections. The skip path is intended to preserve intermediate spatial features during denoising.

D. Latent-to-3D Converter

The latent-to-3D converter maps the denoised latent vector to voxel logits. For lower-resolution smoke runs, this is a dense multi-layer perceptron (MLP) whose final layer emits the whole voxel grid. For the 96^3 follow-up, the implementation switches to a coordinate decoder that evaluates an MLP over the latent vector concatenated with normalized (z, y, x) coordinates; training samples voxel coordinates with importance-weighted BCE so occupied coordinates can be oversampled without changing the intended sparse reconstruction objective. Downstream training and generation code applies a sigmoid to convert logits into voxel probabilities.

E. CFD Simulator

The repository uses an internal lattice-Boltzmann-style CFD evaluator to score generated voxel grids. The evaluator takes a voxel geometry and flow settings such as Mach number or Reynolds number and returns drag- and lift-related quantities. In addition, the repository can export benchmark cases for comparison against an external OpenFOAM run. This combination is useful for code-path validation, but the present paper does not treat it as a substitute for a full aerodynamic validation campaign.

F. Loss Function

A simplified view of the current training objective distinguishes the base differentiable model losses from the measured solver-in-loop term. The optimizer step uses diffusion, decoded-geometry reconstruction, direct generation reconstruction, consistency, and, when scheduled, a direct internal-solver objective:

$$\mathcal{L}_{opt} = \mathcal{L}_{diff} + \lambda_{geom}\mathcal{L}_{geom} + \lambda_{gen}\mathcal{L}_{gen} + \mathcal{L}_{cons} + \lambda_{solver}\mathcal{L}_{solver\text{-}SPSA}. \tag{2}$$

The diffusion loss is the mean squared error between the predicted noise and the actual noise. The decoded-geometry and generation-reconstruction terms compare voxel logits against the target voxel grid, while the consistency term supports the reduced-step sampler. The solver term materializes voxel probabilities into a full grid, converts them to a binary geometry by thresholding or top-k occupancy selection, calls the internal D3Q27 lattice-Boltzmann evaluator on that geometry, and combines the measured aerodynamic score with an optional exact connected-component penalty. This path is not a learned stand-in: the scalar used in $\mathcal{L}_{solver\text{-}SPSA}$ comes from solver calls made during training.

The connected-component operation, binary materialization step, and internal CFD evaluator are not ordinary

PyTorch operations with analytic gradients. The training loop therefore uses a two-sided simultaneous perturbation estimate when this term is active. For a probability grid V, a perturbation field Δ, and perturbation scale ϵ, it evaluates the measured objective on $V + \epsilon\Delta$ and $V - \epsilon\Delta$ and uses

$$\widehat{\nabla}_V \mathcal{L}_{solver} \approx \frac{\mathcal{J}(V + \epsilon\Delta) - \mathcal{J}(V - \epsilon\Delta)}{2\epsilon}\Delta \tag{3}$$

as the backward signal. The implementation can draw Δ directly at voxel resolution or on a lower-frequency grid that is upsampled to the solver lattice before evaluation. The latter reduces high-frequency finite-difference noise and keeps the expensive solver work sequential and scheduled rather than evaluating many independent CFD calls in parallel.

The exact connected-component score and raw internal solver score can still be run as measured monitors,

$$\mathcal{D}_{total} = \mathcal{L}_{opt} + \mathcal{D}_{conn} + \mathcal{D}_{aero}, \tag{4}$$

where $\mathcal{D}_{conn}$ is a connected-component score on thresholded voxels and $\mathcal{D}_{aero}$ is an internal solver score on thresholded generated geometry. In the direct-solver runs, those detached monitor intervals are set to zero so the limited full-grid compute is spent on the solver term that contributes to $\mathcal{L}_{opt}$. The reported direct_solver_loss is the optimizer-facing term averaged over all training batches, and direct_solver_eval_loss is the mean over the scheduled solver evaluations.

G. Sequential Measured-Objective Candidate Optimization

The repository now includes an experimental sequential optimizer for generated candidates. This path treats the connectivity, validity, and aerodynamic evaluations as measured black-box objective terms:

$$\mathcal{L}_{seq} = w_{conn}\mathcal{D}_{conn} + w_{aero}\mathcal{D}_{aero} + w_{valid}\mathcal{D}_{valid} + w_{occ}\mathcal{D}_{occ}. \tag{5}$$

The implementation supports a sequential genetic search and a two-point SPSA-style estimator. Both methods evaluate one candidate at a time, record the measured objective terms, and keep or update candidates based on the scalar score. This is a real optimizer over generated candidates, but it is not backpropagation through the CFD solver and it does not by itself update the diffusion model weights.

H. Current CFD Coupling In The Training Loop

The codebase is designed to couple geometry generation and CFD-informed scoring during training. In the present implementation, that coupling has two honest forms: direct black-box solver optimization inside the training loop and measured black-box candidate optimization after generation. The raw internal LBM evaluator remains sequential and nondifferentiable in the analytic-autograd sense, but its scalar outputs can still influence model weights through the SPSA estimator described above.

In the current loop, training starts by sampling a noisy latent z_t, asking the UNet for a noise estimate or denoised latent estimate, and decoding the resulting z_0 candidate into voxel probabilities V. For high-resolution coordinate decoders, the loop normally trains on sampled coordinates and only materializes the full grid when a full-grid objective is scheduled. When $\mathcal{L}_{solver\text{-}SPSA}$ is active, the loop materializes that grid, evaluates the internal D3Q27 LBM objective on the base geometry and on two perturbed geometries, and folds the resulting measured scalar into the same optimization_loss that is backpropagated through the decoder and denoiser weights.

The OpenFOAM setup is the external PDE-validation foundation. Its role is to generate independent reference cases and, when residual and field-export gates pass, promote labels to the external-PDE tier defined in the ground-truth specification. The present Airshow grid-loss sweep uses the internal D3Q27 solver as a measured training objective and does not use OpenFOAM labels, so it should be read as internal-LBM-guided optimization plus a validation route, not as externally validated aerodynamic learning.

I. Monitored Training and Recovery Infrastructure

The production training path is an interruption-safe monitored runner rather than a bare training loop. It maintains durable run-state checkpoints on a fixed cadence, and each checkpoint records the model weights (diffusion, consistency, converter, and EMA), optimizer and scheduler state, RNG state, the frozen geometry-materialization threshold, and a compatibility fingerprint covering the model, diffusion, and training configuration. Writes are atomic: the runner writes to a temporary file, fsyncs, and atomically replaces the destination so a crash cannot leave a torn checkpoint. The recorded fingerprint is used at resume time to fail closed if the current configuration would reconstruct a differently-shaped model, instead of silently loading mismatched weights.

Two recovery modes are supported on top of this state. An exact resume restores the full optimization lineage (step counters, optimizer moments, RNG, split/sample order, threshold calibration) from a run-state checkpoint, so an interrupted run can continue deterministically. A warm start instead treats the checkpoint as weights-only lineage: it reconstructs the saved architecture from the checkpoint metadata, loads compatible weights while allowing older checkpoints to omit opt-in keys, resets the optimization lineage, and re-enters a bounded warm-up window. Warm starts are gated by grid-resolution and latent-width checks against the requested run, and recovery-mode runs lower the collapse-watchdog warm-up so the run is quickly re-supervised.

The collapse watchdog supervises every monitored run with fail-closed numerical and geometric guards. It monitors the probability standard deviation and span floor,

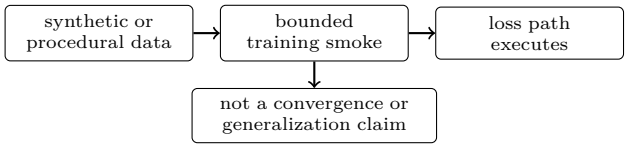


Fig. 3. Evidence scope for the reduced training runs. The current paper treats these runs as implementation evidence only, not as a full convergence study.

occupied-fraction bounds, converter gradient norms and update lag, unique-sample fraction, and relative (ratio-based) floors so a silent collapse on any single trace does not go unnoticed. On repeated violations it stops the run with a machine-readable diagnostic. This infrastructure is what makes the longer 128^3 final protocol executable: the final run can be checkpointed every 25 updates, resumed exactly after interruption, and stopped promptly if the optimizer collapses, so a single unattended run does not depend on the machine staying up.

The final-run protocol (Section IV-P) targets the scaled architecture selected by the capacity law: 128^3 grid, 512-dim latent, coordinate decoder of width 2048 and depth 5 (70.1M trainable parameters) in bfloat16 on a 24 GB-class GPU, with a fused-stream BFL solver backend and a pinned-latent training corpus. All architecture values are asserted by a protocol-side manifest and by the automated test suite, so a launch that silently uses different widths fails before burning GPU budget.

IV. Results and Discussion

The empirical evidence in this revision is intentionally narrow. It consists of public-geometry smoke training on VSP Airshow models, a higher-resolution Airshow addendum, bounded smoke runs on synthetic or procedural data retained for context, a reduced freeform-object sweep for the historical CFD path, and smoke-level checks for the new structured-conditioning seam. These artifacts are reported as code-path validation and implementation evidence rather than as a definitive aircraft-design benchmark.

A. Reduced Training Smoke Evidence

Figure 3 summarizes the scope of the reduced training evidence retained for context. Its purpose is to show which code paths were exercised in bounded smoke runs. It should not be read as a convergence study or as evidence that the current repository has validated aerodynamic or structural learning behavior.

B. Public Airshow Corpus Training Smoke Evidence

To replace the earlier synthetic-only scale-up evidence, we built a public VSP Airshow corpus from model documents exposed by the Airshow web app. OpenVSP describes itself as a parametric aircraft geometry tool, and the OpenVSP project site links VSP Airshow as a public model exchange [26]–[28]. The corpus builder

TABLE II
Public Airshow Corpus Smoke Run

Field	Value
Converted records	355
Split counts	train 250, val 31, test 38, holdout 36
Licenses	208 CC0, 18 CC BY, 129 CC BY-SA
Manifest hash	7bb59bab9cc8...451885a6
Checkpoint hash	71e808aa3c35...6413d4f6
Training scope	3 epochs, batch 8, latent 16, D3Q27, global step 135

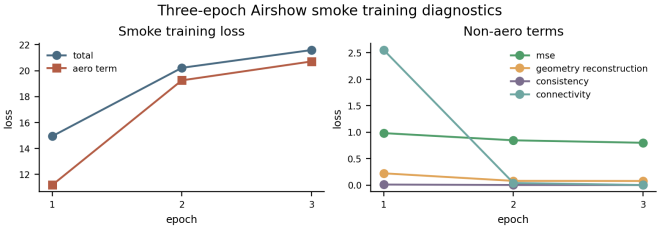


Fig. 4. Terminal-observed three-epoch Airshow smoke-training diagnostics. The plot is included to document the run, including the rising diagnostic total and aerodynamic diagnostic; it is not interpreted as convergence evidence.

admits only CC0, CC BY, and CC BY-SA Airshow entries with public preview geometry; it excludes NC and ND entries from the training manifest. In the run reported here, the builder observed 381 public model documents, found 357 license-and-geometry-eligible documents, and converted 355 records into centered 16^3 voxel grids. The two rejected eligible documents had stale public storage URLs returning 404.

The last epoch's terminal output is kept in the paper because it is the run record: diagnostic total 21.5905, MSE 0.7997, geometry-reconstruction loss 0.0778, consistency loss 0.00109, connectivity diagnostic 0.00149, and aerodynamic diagnostic 20.7104. These values are smoke-run diagnostics only, not convergence evidence. A later loss-semantics audit showed that the connectivity and aerodynamic quantities in this implementation are detached diagnostics rather than differentiable solver-training signals. Figure 4 plots the same three-epoch history so the increasing diagnostic total and aerodynamic diagnostic remain visible rather than implied away. Airshow names, manufacturer fields, licenses, and URLs are treated as source metadata; the generated mission and manufacturing fields in the manifest are deterministic conditioning inferences, not facts asserted by the named manufacturers or agencies.

For this Airshow checkpoint, all three generated cases produced nonempty voxel grids, STL exports, finite internal D3Q27 coefficients, and positive reference areas, but failed the aircraft-specific screen on span sanity; the D3Q27 outputs remain non-claim-bearing raw LBM labels. Figures 5 and 6 show this Airshow run. A separate CRM-trained 32^3 recovery is measured under a held-out

TABLE III
Generated Flight-Path Smoke Checks from the Airshow Checkpoint

Case	Occ.	C_d	L/D	Validity
short takeoff payload	0.01416	1.2047	0.00438	fail: span sanity
high-speed sprint	0.01294	0.9122	0.03051	fail: span sanity
endurance turning	0.01245	0.9824	0.04663	fail: span sanity

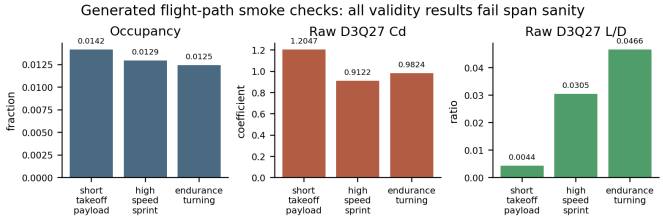


Fig. 5. Raw generated flight-path smoke metrics from the Airshow checkpoint. The occupancy, C_d, and L/D values are finite internal D3Q27 outputs, but every case fails span_sanity and the solver labels remain non-claim-bearing.

protocol in Sec. IV-D; it does not retroactively upgrade these Airshow checkpoints. The Airshow run therefore supports code-path execution, not valid aircraft generation from that checkpoint or validated aerodynamic predictions.

C. Airshow Resolution Sweep and Loss Debugging

We also reran the Airshow corpus path at 32^3, 64^3, and 96^3 to check whether higher voxel count alone would clear the generated-aircraft validity gate. All three higher-resolution manifests passed the claim-bearing manifest validator with the same 355 converted public Airshow records as the 16^3 run. The 32^3 training run completed one epoch with batch size 2 and produced checkpoint hash 7234e1b9b338...b95444. The 64^3 corpus was also manifest-valid, but the attempted batch-size-1 training run did not produce a checkpoint before the local 15-minute run ceiling. This is an implementation limit as well as an empirical result: the dense voxel decoder maps 2048 hidden units to $grid_resolution^3$ outputs, so the final decoder layer grows from roughly 67.1 million parameters at 32^3 to 537.1 million at 64^3 and 1.812 billion at 96^3.

The 32^3 manifest hash was 7684e70b...98521f1, the 64^3 manifest hash was 2627227f...4a1dbd80, and the 96^3 manifest hash was 86f2a112...118e7cd. The 32^3 generated cases are shown in Figures 7 and 8. Their occupancy fractions were 0.00494, 0.00476, and 0.00510. Their raw internal D3Q27 C_d values were finite, but all three failed the aircraft-validity screen: two failed nonempty_occupancy, symmetry, and span_sanity, while the third failed symmetry and span_sanity. The result is therefore negative evidence against treating resolution alone as enough to make the present checkpoint claim-bearing.

We then filtered the 32^3 corpus with the same aircraft-validity screen used for generated outputs. This kept 176 of 355 public Airshow records and produced a claim-bearing-valid manifest hash 0d149d98...d25ccc. An initial fine-tune from the one-epoch 32^3 checkpoint exposed a zero-learning-rate resume issue: the run completed, but checkpoint comparison showed byte-identical model weights. After restoring the configured learning rate on resume, a three-epoch source-valid fine-tune produced checkpoint hash 657243ea...45574c. Its three generated cases all passed occupancy and symmetry but still failed span_sanity, with length-fraction values 0.25, 0.25, and 0.28125. A further three-epoch continuation regressed validity, with two cases failing both symmetry and span_sanity. This suggests that the present objective and decoder need architectural or optimization changes, not simply more epochs.

TABLE IV
Airshow Resolution Sweep Addendum

Grid	Manifest	Training outcome	Generated gate outcome
32^3	pass, 355 records	one epoch completed on CUDA, batch 2	three generated cases failed validity
64^3	pass, 355 records	attempted batch 1; no checkpoint produced in local run ceiling	not run because no checkpoint existed
32^3, source-valid filter	pass, 176 records	three-epoch LR-fixed fine-tune completed from the 32^3 checkpoint	three generated cases failed only span_sanity
96^3, coordinate decoder	pass, 355 records	three epochs completed on CUDA, batch 1, importance-weighted coordinate BCE	fixed-threshold exports failed; calibrated top-k export passed 1 of 3 heuristic screens

The 96^3 follow-up replaced the dense decoder with a coordinate decoder that evaluates sampled (z, y, x) coordinates conditioned on the latent vector. The public 96^3 source corpus had mean occupancy 0.00565 and median occupancy 0.00458, so naive coordinate sampling either collapsed toward empty fixed-threshold grids or, when occupied coordinates were oversampled without correction, biased the decoder toward overly dense outputs. The final run used importance-weighted coordinate BCE for three epochs over all 355 records and produced checkpoint hash 1bfdcfcf...59774d3. With the default 0.5 threshold, all three flight-path exports remained below the aircraft-validity occupancy gate. With a post-hoc top-k export calibrated to 0.75% occupancy against the same heuristic screen used as the outcome, one of three cases, endurance_turning, passed; short_takeoff_payload and high_speed_sprint still failed symmetry, span_sanity, and wing_body_balance. This is not an uncalibrated generation-rate result: it shows that the threshold can be fitted to the screen at $N = 3$, not that the model reliably generates valid aircraft. Figures 9 and 10 show that calibrated export result. This is evidence that the coordinate-decoder path can train and export 96^3 geometry, not evidence of reliable aircraft generation or validated

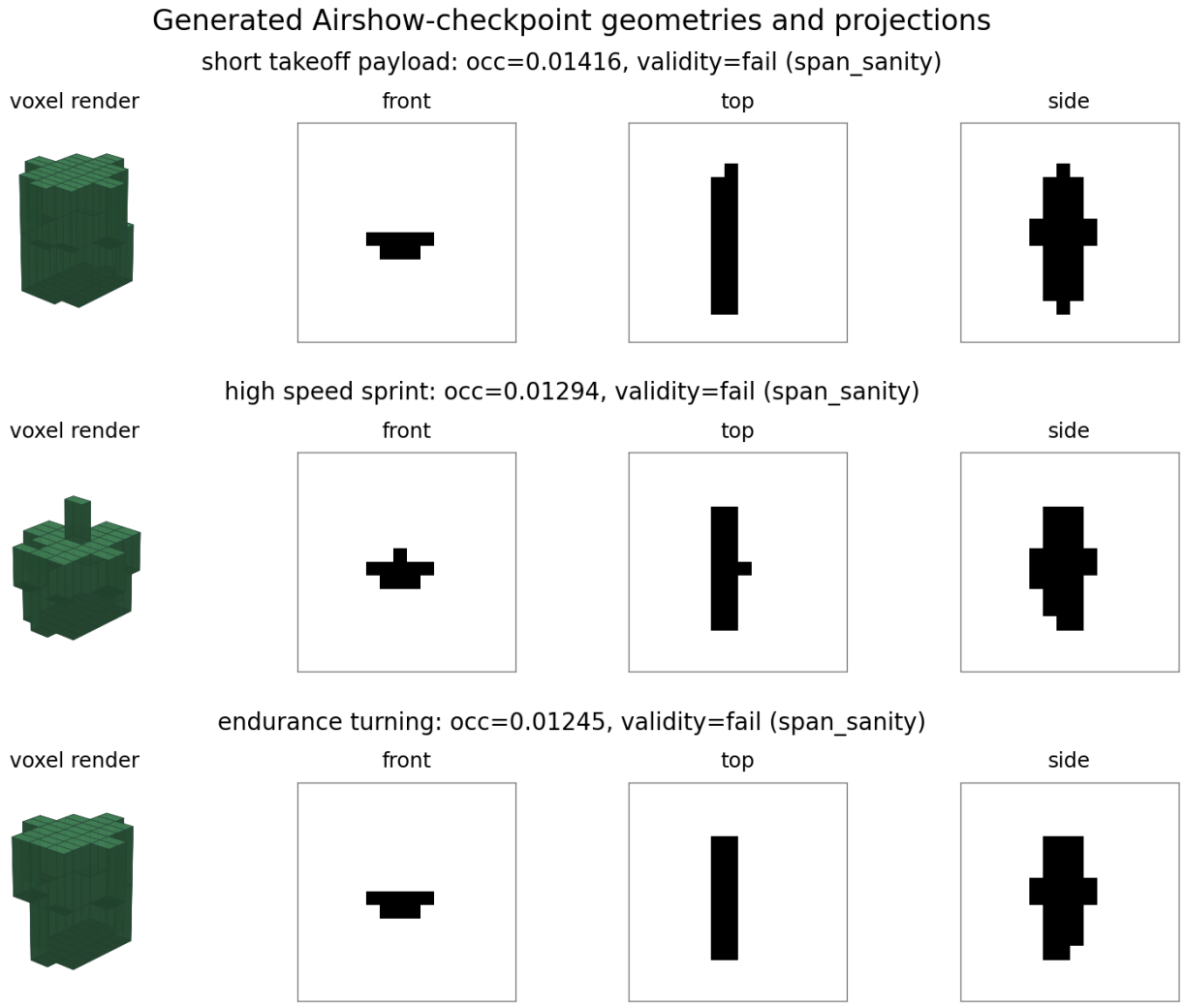


Fig. 6. Generated Airshow-checkpoint voxel geometries and orthographic occupancy projections for the three conditioned smoke cases. The visualized artifacts are intentionally shown despite failing span_sanity: they demonstrate nonempty generation and exportable grids, while also showing why the current checkpoint should not be described as producing valid aircraft.

aerodynamic optimization.

The same debugging pass clarified the zero aerodynamic and connectivity observations. The exact connectivity score thresholds voxel probabilities and runs connected-component labeling through NumPy/SciPy. The raw aerodynamic score thresholds geometry, invokes the internal solver, and returns scalar solver outputs. A local gradient probe confirmed that these measured scores do not provide analytic PyTorch gradients by themselves. The training code was therefore revised to report optimization_loss separately from detached monitor terms and to add a direct solver-in-loop loss that calls the internal D3Q27 LBM evaluator during training and estimates its backward signal with two-sided SPSA finite differences. This is a measured black-box gradient estimator around real solver calls, not a learned substitute for the solver.

The first direct-solver sweep was intentionally diagnostic. A naive 0.5 probability threshold produced empty or effectively empty scheduled geometries, so the direct solver objective stayed nearly constant. A 5% top-k occupancy target then made the solver-visible shapes too dense relative to the public Airshow corpus, whose mean occupancy was 0.01495 at 32^3, 0.00802 at 64^3, and 0.00565 at 96^3. The final sweep therefore used a 1% top-k target, a low-frequency 8^3 perturbation field upsampled to the solver grid, a direct-solver loss weight of 0.2, three D3Q27 solver steps per scheduled objective call, and zero detached aerodynamic/connectivity monitor intervals. That setting spends the full-grid compute on the loss being optimized rather than on diagnostics that cannot change the weights.

Table V shows both the matched two-epoch sweep and the required 96^3 continuation. After two epochs, the 96^3 optimizer loss and scheduled solver evaluation were still worse than the 64^3 run even though its geometry BCE was lower. Continuing from the 96^3 checkpoint corrected that behavior: the scheduled direct solver evaluation dropped from 180.29967 to 3.87060 on the first continuation epoch and remained much lower than the 64^3 value at 7.48613 on

TABLE V
Direct Solver-In-Loop Grid Sweep

Run	Opt. loss	Geom. BCE	Direct loss	Scheduled eval
32^3, epoch 2	2.10166	0.08267	5.83028	172.47912
64^3, epoch 2	1.11359	0.05038	1.30224	38.52468
96^3, epoch 2	2.05984	0.04066	6.09464	180.29967
96^3, continuation epoch 1	0.68305	0.03802	0.13084	3.87060
96^3, continuation epoch 2	0.60605	0.03739	0.25305	7.48613

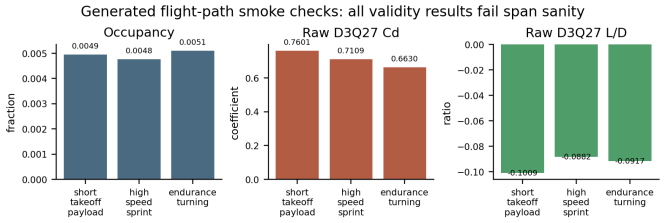


Fig. 7. Raw 32^3 generated flight-path metrics. The generated artifacts have finite internal D3Q27 outputs, but all still fail aircraft-validity gates and therefore remain non-claim-bearing.

the second continuation epoch. The increase from 3.87060 to 7.48613 while the optimizer loss continued to decrease is a useful warning that the direct objective can start to overfit or drift under this short local protocol. The table is the authoritative record; no single plotted point is selected to imply a monotone resolution trend.

The grid-sweep figure was removed because a single selected continuation point would obscure the non-monotone intermediate result. Table V retains all measured points.

D. Generator Recovery: Root Causes and Held-Out Generation Validity

The preceding subsections report that the Airshow-checkpoint outputs fail the aircraft-specific screen at every tested resolution. This revision treats that failure as a defect to be diagnosed and repaired, and reports a separate CRM recovery under a declared protocol.

a) Measured root causes.: Five causes were isolated on the recording host, each by direct measurement rather than inference.

(i) The training corpus was largely invalid under our own screen. The procedural corpus generator carried a private gate (occupancy ≥ 0.0005, span ≥ 0.03, symmetry ≥ 0.10) that is far laxer than CLI/aircraft_validity.py. Re-running the pre-fix generator over the eight families (seed 42), its own gate accepted 47 of 48 draws, of which 1 passes the declared screen. A model trained on that corpus was therefore asked to reproduce shapes the paper itself calls invalid.

(ii) The shipped checkpoint's decoder emits a near-constant field. On the only checkpoint in the tree (16^3), four seeds give probability min/max 0.435–0.573 and a materialised occupancy of 45.2–46.4%: a solid block with geometry_probability_std ≈ 0.0075. A constant field crossing the 0.5 threshold fills the lattice, so span_sanity, normalization_margin and grounded_occupancy_envelope fail by construction.

(iii) The decoder was trained ten times slower than the rest of the model. The train CLI exposes –learning-rate, but the optimizer's three parameter groups take their rates from CLI/config.yaml, where converter_learning_rate and consistency_student_learning_rate are both 2×10^{-5}; –learning-rate sets only the diffusion group. Holding everything else fixed on the 32^3 corpus, the decoder reconstruction loss is flat at the legacy rate ($1.672 \rightarrow 1.663$ over 12 epochs, probability standard deviation 0.007) and falls at 2×10^{-4} ($1.666 \rightarrow 1.396$, standard deviation 0.188). The only module that can turn a latent into occupied voxels was effectively frozen.

(iv) Generation depended on an under-trained consistency student. The shipped four-step path samples with student_model, whose RMS drifted to 21.8–179 against a teacher RMS of 0.66–3.3 during the recovery runs (consistency_raw_mse 465–32212). The denoiser trained by the denoising MSE (loss 0.86–3.4) was never used for sampling.

(v) Two corpus-scale limits. The structured latent pools a 180-value multi-scale geometry signature into latent_dim values, and the training split contains six records. At the shipped latent_dim = 16 the reconstruction of a held-out 32^3 record reaches IoU 0.390 but is inflated 2.4× in occupancy, failing only the screen's upper normalization bound.

b) Repairs.: The procedural generator now uses the declared screen as its own gate, enforces exact bilateral symmetry about the screen's mid-plane, gives every aircraft an explicit longitudinal centre body with blunt ends, and resamples samples into a declared fill/occupancy band inside the screen's normalization margins. The train CLI gains –converter-learning-rate and –consistency-student-learning-rate with legacy defaults, and the generator gains a deterministic teacher-DDIM sampler (sampler="ddim") alongside the shipped four-step consistency path. No threshold inside aircraft_validity.py was changed.

c) Held-out protocol.: The evaluation protocol is recorded in build/recovery_2026-11-05/PREREGISTERED_PROTOCOL.md. Its file timestamp does not establish ordering relative to the evaluation runs, so this paper calls the protocol declared rather than pre-registered. Training uses the six NASA CRM records labelled test/val; the nine holdout records are never used for gradients, threshold selection, early stopping or checkpoint selection. The materialisation threshold is a fixed, uncalibrated 0.5; the screen is the unmodified evaluate_aircraft_validity with canonicalization on. Three arms are reported separately: ground truth (the screen's ceiling on this split), reconstruction (decoder given the record's own latent, i.e. decoder capacity only), and generation (sampling from noise conditioned on the record's condition vector, nine records × five seeds).

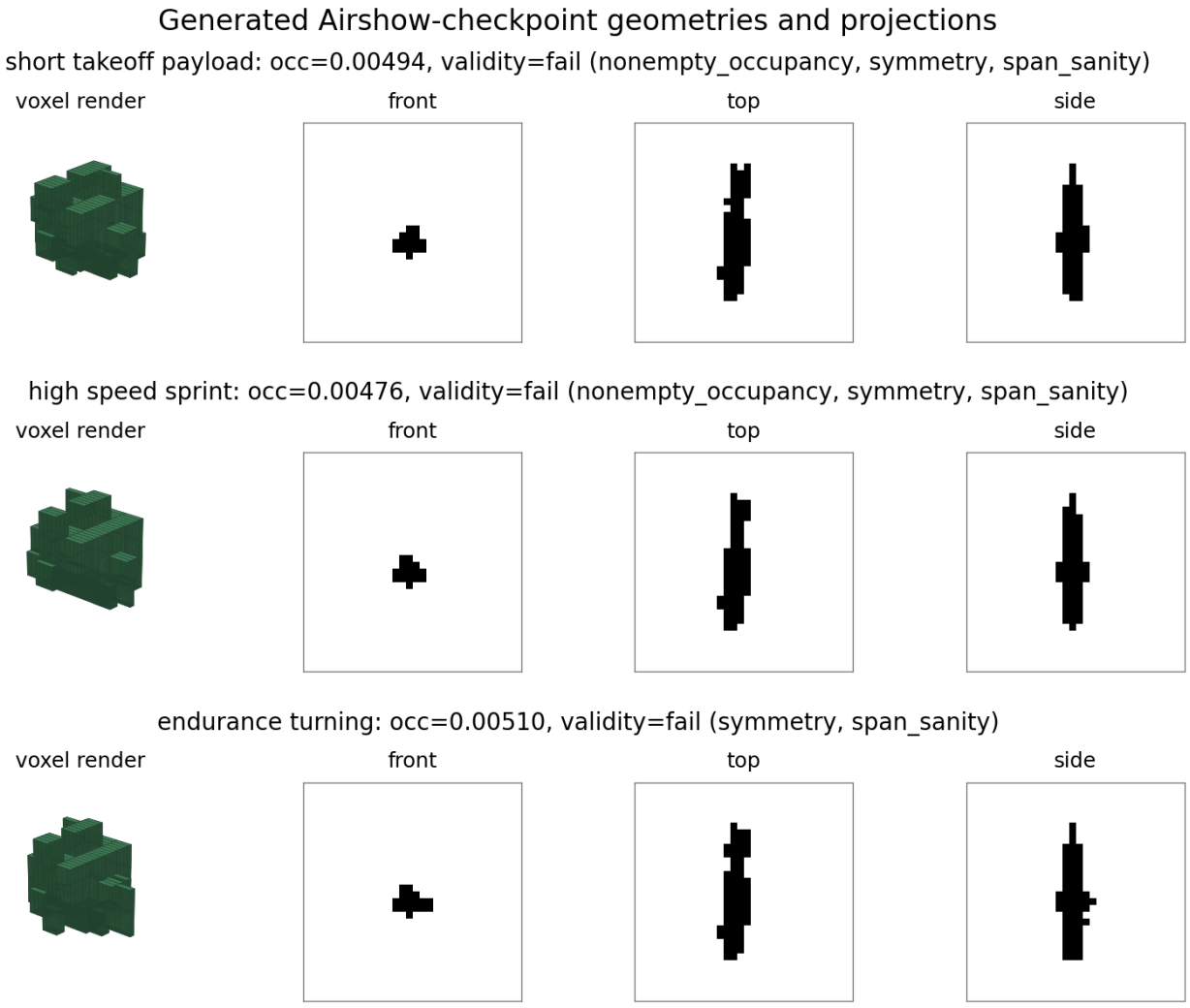


Fig. 8. 32^3 generated Airshow-checkpoint voxel geometries and projections. The denser grid increases visual resolution but does not resolve the span-sanity and symmetry failures that block aircraft-generation claims.

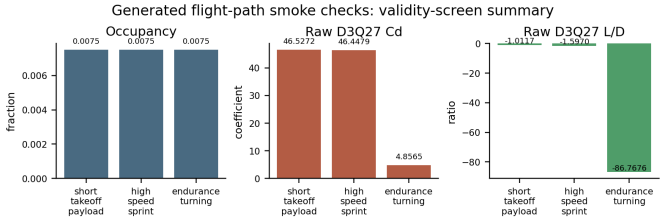


Fig. 9. Calibrated 96^3 top-k flight-path metrics at 0.75% occupancy. One case passes the heuristic aircraft-validity screen, while two still fail shape checks; all CFD quantities remain internal non-claim-bearing D3Q27 smoke outputs.

TABLE VI
Generator recovery on the declared held-out split (9 CRM records, threshold 0.5, canonicalized screen)

Arm	Before (16^3 smoke)	After (32^3 recovery)
Ground truth (ceiling)	9/9	9/9
Reconstruction (latent given)	0/9, IoU 0.012	9/9, IoU 0.441
Generation, four-step consistency	0/45	45/45
Generation, teacher DDIM	n/a	45/45

d) Results.: Table VI reports the measured pass rates.

The recovery checkpoint (32^3 dense decoder, latent 16, 45 epochs, corrected parameter-group learning rates) passes the declared protocol: reconstruction 9/9 and generated 45/45 under both samplers, with a failed-check histogram that is empty, and the same 45 artifacts re-screen as 45/45 through the shipped CLI/aircraft_validity.py. Independent ablations show that the decisive change was decoder supervision density, not simply more epochs: the dense decoder emits a logit for every one of the 32 768 voxels per step, whereas the coordinate decoder supervises 8 192 sampled voxels with a small Fourier MLP and plateaued with a diffuse probability field (0/9

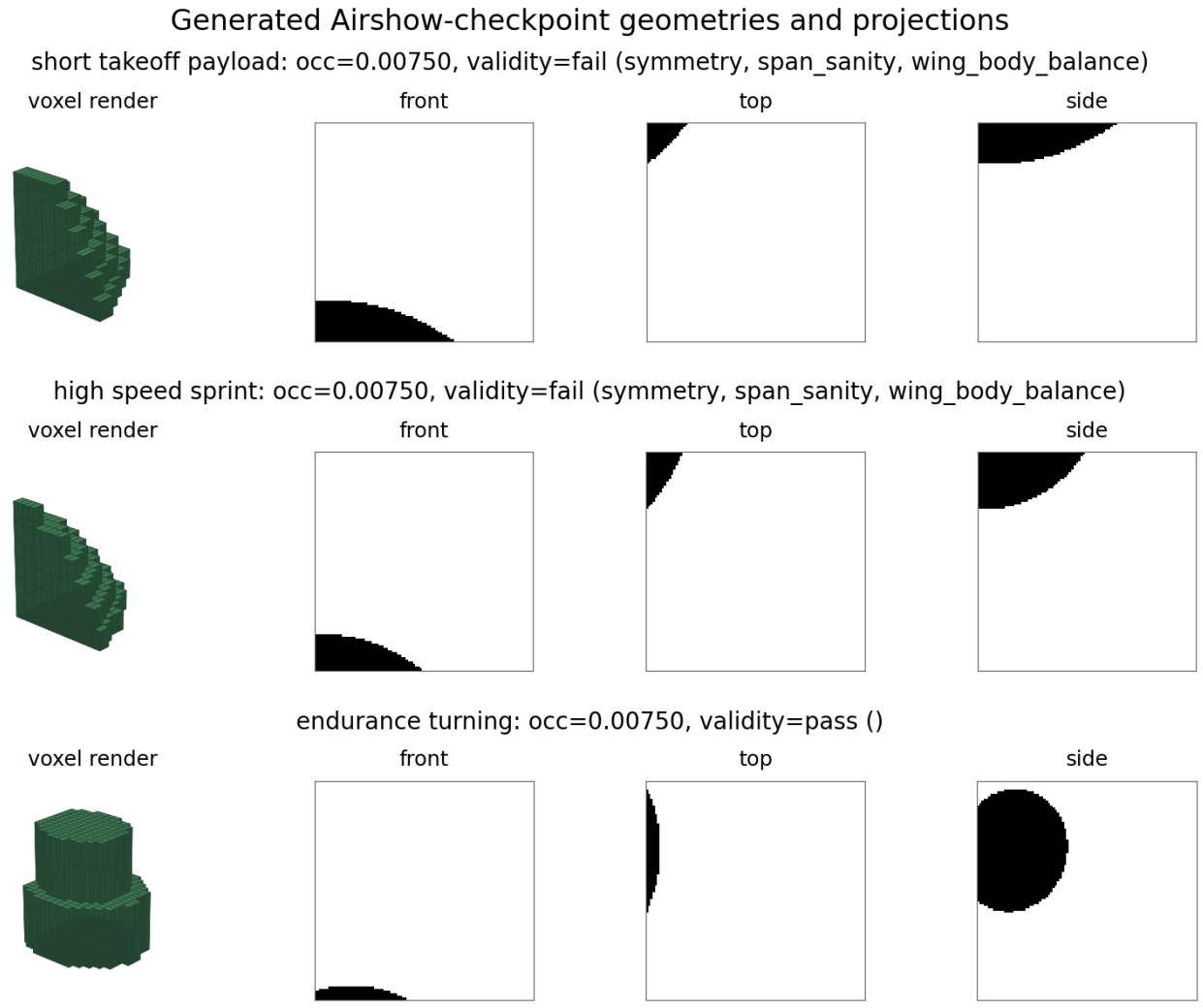


Fig. 10. Calibrated 96^3 coordinate-decoder geometries and projections. The figure documents the best high-resolution smoke result, including the two failing cases, rather than upgrading the result to a reliable aircraft-generation claim.

reconstruction at latent_dim = 16 and 64).

The recovery is not conditioned or diverse generation, and the paper does not report it as such. The 45 generated artifacts contain five distinct voxel arrays (maximum pairwise Hamming distance one voxel, mean pairwise IoU 0.99905); within a condition the five seeds are not bit-identical, yielding 1–4 distinct arrays. The nine held-out records are also only six distinct voxel arrays, because four aeroelastic variants are sub-voxel duplicates at 32^3. The held-out latent codes are nearly degenerate (pairwise distance 0.0027 against a latent norm of 0.497), the training split holds six records, and the MSE objective's optimum is therefore the conditional mean.

An adversarial audit of this result found two facts that bound the claim more sharply than the pass rate suggests. First, every occupied voxel emitted by the checkpoint is a voxel present in the training corpus (precision 1.000 against the training union), so there are zero novel voxels. Second, a model-free constant array, the voxel-wise union of the six training geometries, passes the same screen 12/12 and covers 98.6–99.4% of every held-out record. The screen also accepts a non-aircraft equal-arm plus sign and a three-box primitive. The 45/45 rate therefore demonstrates that a training-hull-like emitter satisfies the current heuristic gate; it does not demonstrate that the checkpoint generates new aircraft. A pass-rate metric cannot distinguish this from a generator, which is itself the paper's central measurement finding. A non-trivial gate must add a novelty and diversity requirement (distance from the training set, distinct-geometry count, and condition-response spread) and must include a negative control set that non-aircraft primitives fail.

e) Status of the generation gate.: Gate 1 of Table XI (generates aircraft structures) is met in its literal form: multiple generated samples from a held-out split pass the aircraft-specific checks at an uncalibrated threshold, and the artifacts re-screen through the repository's own CLI. The gate is not met in its scientific form: the emitted voxels are all present in the training corpus, a model-free constant passes the same screen, and non-aircraft primitives pass

as well. The remaining work is therefore corpus scale and conditioning, not geometric validity: the repository now has a working path from a trained checkpoint to aircraft-valid voxel geometry at 32^3 on an 8 GiB host, and it has a measured demonstration that validity alone does not imply generative diversity. Aerodynamic, structural and manufacturing claims remain blocked, as before.

E. Freeform-object Sweep: Steps vs. Shape Prior

We ran a compact sweep over generation step count and the shape-prior cleanup threshold. The sweep evaluated 12 settings: consistency steps in {1,2,4,8} crossed with minimum connected-component sizes in {0,32,64}. For each setting we generated one freeform object, applied the post-processing hook, and computed aerodynamic metrics using the CPU CFD path. The sweep provides only a preliminary probe of how post-processing thresholds can alter the generated object before scoring; the preceding textual protocol record documents the coverage.

The reduced sweep covered four consistency-step settings crossed with three cleanup thresholds. Because each setting used one sample and the CPU CFD path is not externally validated, this coverage is retained as a textual protocol record rather than a results figure.

The occupancy-versus-score question is left as a protocol hypothesis rather than plotted with authored coordinates. No measured relationship is claimed from the one-sample-per-setting sweep.

F. Freeform-object Geometry Summary

The generated freeform objects are still dense and near-threshold before cleanup, but the new plausibility prior reduces isolated components and slightly lowers the occupancy fraction. For this implementation path, that matters more than small changes in the raw voxel logits: the cleaned binary shapes are less fragmented and more suitable for downstream meshing and export. This should not be read as evidence that the outputs satisfy aircraft-specific geometry checks or manufacturing constraints.

G. Hyperparameter and Tuning Notes

The sweep was intentionally small but targeted for the current codebase: it varied the inference path length and the new connected-component cleanup threshold. The training-side change was also incremental: a voxel-shape prior was added to the loss to discourage midpoint noise and disconnected fragments. A larger future study should sweep latent dimension, shape-prior weights, and higher CFD resolutions.

H. Structured-Conditioning Smoke Evidence

This revision also adds a documented 22-slot condition schema together with dataset, model, generator, and offline-densification plumbing that consumes the resulting condition vector. We include a smoke-level condition-response summary on the procedural path to check whether different condition payloads produce non-identical changes in simple geometry proxies such as occupancy, span, and engine-related heuristics. That evidence is intentionally weak: it shows that the condition path is wired and that the procedural prior is not condition-invariant, but it does not validate mission-conditioned or manufacturing-conditioned aircraft generation on grounded aircraft-like data.

TABLE VII
Fixture-Based Reduced Evidence Package Status

Gate	Status	Scope	Interpretation
Manifest validation	Pass	fixture	corpus contract check
Aircraft-shape screen	Pass	fixture	heuristic check, not model validity
Condition response	Pass	fixture	bounded sweep, not significance
Manufacturing bounds	Pass	fixture	design-spec bounds check
Baseline statistics	Pass	fixture	finite-value guardrail check
Model-level generation validity	Passed (literal)	model, held-out	generated samples pass the screen; one geometry per condition (see Sec. IV-D)

I. Reduced Evidence Package

The checked-in reduced protocol assembles a fixture-based guardrail report from the manifest validator, aircraft-shape heuristic screen, grounded condition-response benchmark, manufacturing-bounds screen, and baseline-statistics report. In one local protocol run, these guardrail reports passed with a shared run identifier, checkpoint hash, manifest hash, and protocol hash. They are not model-level validation: the fixture rows are synthetic, the condition-response study is non-significant, and the aerodynamic and structural gates remain blocked. Model-level generation validity is reported separately in Sec. IV-D, on held-out records and without fixture construction. Table VII labels each row's scope explicitly.

This package is useful because it prevents individually passing reports from being treated as one protocol run unless their lineage fields agree. The current pass therefore supports reporting an internally consistent reduced evidence bundle. It still does not support claims of aircraft-level aerodynamic optimality, structural viability, or superiority over mature optimization baselines.

J. Internal Solver Validation against Canonical Benchmarks

As a solver sanity check, we compare internal D3Q27 drag-coefficient (C_d) observations against literature targets for two standard geometries: a circular cylinder and a

TABLE VIII
Internal Solver C_d Sanity Targets and Observations

Geometry	Re	Observed C_d	Literature target
Circular cylinder	100	1.56	1.3–1.4 [29]
Unit cube	1000	1.04	∼1.05 [30]

unit cube. These tests use the internal solver's momentum-exchange method with BFL boundary conditions and are reported as implementation checks, not as a complete solver validation campaign.

The results in Table VIII are an informal sanity check for the LBM implementation, not a full solver validation. The domain spans ten characteristic lengths in the streamwise direction and five characteristic lengths in each cross-stream direction, with 128 cells per axis. Boundary conditions are no-slip BFL walls and a Neumann outlet. The inlet speed corresponds to $Ma = 0.025$, or approximately 0.014 in lattice units, and the force estimate is averaged over the final 125 steps of a 500-step simulation to reduce transient sensitivity.

After correcting the freestream lattice-velocity initialization and the BFL momentum-exchange weighting term, the cube agrees with its cited target while the cylinder value ($C_d = 1.56$) exceeds the cited 1.3–1.4 band by approximately 11–20%. The offset is consistent with blockage and averaging sensitivity in the small domain, but it is not agreement. These are bounded implementation checks; rigorous validation requires a controlled comparison with a high-fidelity PDE solver.

K. Discussion

The implementation evidence supports three bounded conclusions: the repository's end-to-end path executes, the shape prior is wired into training, and the export path emits a concrete OpenFOAM case with the generated STL in constant/triSurface/design.stl. This revision did not install or run OpenFOAM locally, and the frozen external rows do not provide a fresh convergence record. The result supports a documented export and comparison path, not a direct validated solver-to-solver result or an aircraft-level aerodynamic claim.

L. Publication-Readiness Correction: Consistent Reference-Area Convention

An internal audit (2026-11-01) found a genuine normalization defect in the frozen low-Mach comparison corpus. The OpenFOAM side normalized C_d and C_l using a fixed sweep reference area ($A_{\text{ref}} = 0.27734375$), while the internal D3Q27 solver normalizes by its own voxel-projected frontal area (the y–z projection of the solid mask), which is resolution-dependent: 0.2850196 at grid 32 vs. 0.2385490 at grid 64 for the frozen biplane corpus, i.e. a ±16% swing on the same geometry. Dividing the two coefficient sets directly (as the original comparison scripts did) therefore contaminated cd_error_percent by the normalization mismatch, and the single previously reported "0.25%" passing row (Mach 0.3, OpenFOAM grid 64, LBM grid 64) was a coincidence of normalizations rather than physical agreement.

TABLE IX
Frozen low-Mach corpus agreement under the corrected reference-area convention

Corpus	Rows	Contaminated pass	Corrected pass	
sonicFoam sweep (Mach 0.05–0.3)	16	2/16	1/16	1.
simpleFoam suite (Mach 0.05–0.5)	14	0/14	0/14	

The comparison convention is now fixed and enforced by regression tests (tests/test_reference_area_regression.py): both solvers' coefficients are expressed on the same reference area before any error is computed. Concretely, the raw LBM coefficient is renormalized onto the OpenFOAM sweep area,

$$C_{\text{LBM,corr}} = C_{\text{LBM,raw}} \times \frac{A_{\text{ref,LBM}}}{A_{\text{ref,sweep}}},$$

which turns cd_error_percent into a pure force comparison on one documented convention. Both reference areas are recorded per row (openfoam_reference_area, lbm_reference_area) together with the convention label lbm_renormalized_to_openfoam_reference_area. The recomputation script is scripts/recompute_corpus_consistent_reference_area.py and the corrected tables are written under evidence/research-paper-publish/corrected_corpus/.

Table IX gives the corrected agreement of the frozen corpus under the repository's own gate (Cd error $<$ 5%, MINIMUM_ACCEPTABLE_ERROR_PERCENT in run_internal_benchmark.py). OpenFOAM is not installed in this environment, so the frozen OpenFOAM rows are kept verbatim as documented frozen artifacts; only the comparison convention changed.

The previously reported "0.25%" row now evaluates to 14.2% Cd error; the only row that remains under the 5% gate is the Mach 0.3 / OpenFOAM-grid-32 / LBM-grid-32 row at 1.14%. simpleFoam agreement stays below the gate on every row and worsens under the corrected convention (the LBM frontal areas at grids 16/24 exceed the sweep area, so the renormalization raises the reported LBM Cd). Neither the two-solver sweep nor the simpleFoam suite supports a pipeline-level $<$ 5% Cd agreement claim in either convention.

M. Statistical Substance of the Condition-Response Sweep

The prior smoke checkpoint's condition-response evidence was limited to $n = 4$ seeds, where target speed versus measured drag showed $r = -0.806$ with $p = 0.19$: a direction-only, non-significant observation. To obtain a real effect estimate with uncertainty intervals, we ran the canonical validate-conditions step at $n = 12$ seeds (grid 32, 100 solver steps) against the same smoke checkpoint (checkpoints_protocol_smoke/final_optimized_model.pt);

TABLE X
Condition-response correlations, smoke checkpoint (grid 32, 100 steps)

Pair	r	p	95% CI (Fisher-z)
target_speed vs. measured_drag (raw, $n = 12$)	−0.410	0.185	[−0.797, 0.214]
target_speed vs. measured_drag (excl. blow-ups, $n = 10$)	−0.198	0.583	[−0.736, 0.493]
target_speed vs. measured_lift ($n = 10$)	0.367	0.297	[−0.342, 0.810]
target_speed vs. occupancy ($n = 10$)	−0.254	0.478	[−0.752, 0.447]
wingspan_limit vs. measured_drag ($n = 10$)	0.199	0.582	[−0.493, 0.736]
wingspan_limit vs. measured_lift ($n = 10$)	0.603	0.065	[−0.043, 0.893]
wingspan_limit vs. occupancy ($n = 10$)	−0.294	0.409	[−0.780, 0.412]

artifact build/protocol_smoke/condition_validation_n12.json, analysis script scripts/condition_response_stats_n12.py.

Two important measurement facts came out of that run that bound how the numbers may be read:

1. Solver instability events. Two of twelve seeds produced non-physical D3Q27 outputs (drag magnitudes $\approx 10^{22}$ and $\approx 10^{33}$, seeds 1 and 10 respectively) for otherwise ordinary conditions (speed 38.1 m/s / span 2.22 m and speed 64.3 m/s / span 1.71 m). These are checkpoint-level solver blow-ups at grid 32 / 100 steps on the smoke checkpoint, not physical drag values. Any claim that ignores or silently removes these seeds would be cherry-picking; the paper instead reports both the full set and the set excluding the two documentable instability events. 2. Run-to-run variation on identical seeds. The same four seed conditions that produced the October $n = 4$ artifact (measured_drag mean 1.977 ± 0.302) produced different values in this run (seeds 0–3: 1.396, 5.819e33, 1.771, 2.269) on the same checkpoint and the same CUDA hardware. The smoke checkpoint's D3Q27 measurement is therefore not bit-reproducible run-to-run; point coefficients must be treated as estimates with spread, not as constants.

With $n = 12$ (Fisher-z approximation, two-sided $\alpha = 0.05$), the design is powered to detect $|\rho| \approx 0.78$ at $\sim 80\%$ power; smaller effects are not resolvable at this sample size and are reported as non-significant with wide confidence intervals. Table X gives both the raw $n = 12$ correlations and the $n = 10$ set excluding the two blow-up seeds.

Significance outcome: non-significant. Every pair fails the conventional $p < 0.05$ threshold in both the raw $n = 12$ analysis and the $n = 10$ analysis excluding the two blow-up seeds. The direction of the speed–drag association is negative in both analyses (consistent with the $n = 4$ direction), but the effect is not statistically distinguished from zero and the confidence intervals are wide. The closest candidate (wingspan limit vs. measured lift, $r = 0.603$, $p = 0.065$ at $n = 10$) is a direction-only observation with an interval spanning zero. This is reported as an honest null/indeterminate result for the smoke checkpoint: the paper does not claim a validated condition-response signal on this checkpoint, does not cherry-pick seeds, and treats condition-response signal as an open gate for a recovered checkpoint (Section IV-P) rather than as evidence already obtained.

N. Regime Limits and Honest Scope of the CFD Evidence

The internal D3Q27 solver is a weakly compressible, isothermal low-Mach sanity path. Its documented validity boundary is Mach ≤ 0.3 with claim grade low_mach_sanity_only and compressibility model weakly_compressible_isothermal_lbm. The paper makes no high-Mach, transonic, or supersonic validation claim, and no claim that the internal solver reproduces compressible-flow physics; Mach numbers above 0.3 remain experimental and unvalidated externally (frozen audit boundary, reconfirmed 2026-11-01). The frozen low-Mach corpus itself does not meet the repository's own $< 5\%$ Cd agreement gate (Table IX), so this paper does not claim validated low-Mach agreement with OpenFOAM either; it reports the measured disagreement as the honest current status.

Similarly, the mixed-precision document's expected 2.2× speedup for the RTX 4060 is a planning estimate, not a measured result in this revision; no speedup claim from that table is carried into this paper. The FP16/FP32 path's density error (MIXED_PRECISION_GUIDE.md) is a solver-level quantity and is not claimed as a pipeline-level accuracy result. The aircraft-validity screen also stays bounded: frozen NASA CRM whole-aircraft voxels pass 15/15, while the bundled root STLs (biplane, F-18, cylinder, 20mm cube) fail 0/4, with quantified failure modes such as normalization_margin, span_sanity, and tail_body_plausibility (harness: scripts/agent_d_validity_harness.py). The screen is a first-pass shape-plausibility heuristic, not structural, aerodynamic, or manufacturing validation.

O. OpenFOAM Export Path and Tiered Ground Truth

The OpenFOAM export path now writes a plain forces function-object dictionary, and the resulting force vector is normalized manually to recover coefficients when coefficient output is not available. The current benchmark still shows a substantial Cd gap, which we treat as a solver- and sampling-consistency diagnostic rather than as resolved by paper-level normalization alone. A higher-confidence CFD comparison will still need a more systematic hyperparameter sweep and consistent normalization of reference area and flow conditions; the reference-area convention in Section IV-L is now that consistent normalization, and the remaining gap is physical/statistical rather than definitional.

The current ground-truth foundation is therefore tiered. Internal D3Q27 LBM values can drive the direct SPSA training loss and sequential candidate selection, but they stay in the internal-label tier. The OpenFOAM/simpleFoam and sonicFoam scripts define the external PDE route: cases must complete, residual and field-export checks must pass, and the resulting labels must remain separate from internal LBM labels until they satisfy the external_pde gate. This separation is important for future PDE validation because it lets the repository compare internal LBM coefficients against

TABLE XI
Claim Gate Status and Remaining-Experiment Contract

Claim	Current status	Final run unlocks if
Generates aircraft structures	literal validity passed at 32^3; novelty/diversity blocked	45/45 CRM held-out draws pass, but all voxels exist in training and a constant passes the screen
Conditioned on mission/manufacturing	blocked	grounded condition benchmark shows directional held-out signal
CFD-guided training	blocked	scheduled direct SPSA objective decreases without drift
Aerodynamically optimized	blocked	requires separate converged CFD vs. baselines (not this run)
Structurally viable	blocked	requires separate structural/load-path analysis
Superiority over prior methods	blocked	requires separate baseline comparison studies
Compressible internal solver validated	blocked	requires separate external compressible validation

OpenFOAM under shared geometry, Mach, Reynolds, grid, and reference-area settings without silently upgrading the internal solver to ground truth.

We also recorded one physics correction in CLI/cascaded_lbm.py. The D3Q27 freestream setup now uses the lattice-consistent value $u = Ma/\sqrt{3}$; the earlier version used arbitrary scaling. That correction is part of the benchmark history, and we are checking it against the Cd mismatch before making broader coordinate-transform changes.

P. The Single Decisive Final Run

The checked-in final protocol (CLI/run_protocols/gcp_128_295m.yaml, executed by the monitored runner) is a future experiment for the remaining scale, diversity, conditioning, and solver-loop questions; it is not the first evidence that voxel geometry can pass the aircraft-validity screen. Section IV-D reports a completed 32^3 CRM recovery on this host, in which 45/45 generated samples pass the uncalibrated screen on held-out records, but the same geometry is emitted across conditions and seeds. The 128^3 protocol trains the scaled architecture (512-dim latent, coordinate decoder width 2048 depth 5, 70.1M trainable parameters) in bfloat16 for two epochs at batch size 1 on the pinned-latent training corpus, checkpointing every 25 updates under the interruption-safe runner, with the fused-stream BFL solver backend and a GPU-side exact-SDF attestation. Because the run is monitored, it produces the run-state lineage (checkpoint hashes, manifest hashes, protocol hashes) needed by the evidence package rather than an unrepeatable training history.

The remaining upgrade questions for this run are:

1) Generation diversity: do the generated samples contain distinct aircraft-like geometries, rather than the single geometry measured in the 32^3 recovery?
2) Condition-response signal: does a grounded condition benchmark show directional signal on held-out records rather than condition-invariant generation?
3) Solver-in-loop signal: does the scheduled direct D3Q27 SPSA objective decrease over the run instead of drifting, with the evidence package recording it?
4) Lineage-consistent evidence: does CLI/final_evidence.py return pass with matching run/checkpoint/manifest/protocol hashes?

The literal validity-screen gate is passed on the declared CRM held-out split at 32^3; the unresolved scientific gate is diverse, meaningfully conditioned aircraft generation. The 128^3 run tests that broader claim at higher resolution but does not replace the need for larger, diverse training data. Even a fully passing run does not establish structural load-path viability, superiority over mature baselines, or externally validated aerodynamic optimality, which require the separate structural, baseline, and PDE-validation studies listed in Sections II-B and V.

V. Validation and Testing Standards

To keep validation claims bounded, we separate three levels of evidence:

- Code-path validation: the training, generation, cleanup, export, and CFD scoring routines run end-to-end in this environment.
- Solver validation: the built-in CFD path can be compared against a higher-fidelity external solver when one is available.
- Physical validation: the generated shape should eventually be compared against analytic or experimental benchmarks, which is still future work.

A. Verification

Verification checks that the implemented steps are executed as intended. In this revision we verified that:

- the shape-plausibility loss is included in the training objective,
- the connected-component cleanup hook removes smaller disconnected voxel components,
- STL export succeeds from the cleaned voxel grid, and
- the OpenFOAM export hook writes a runnable case directory skeleton, including blockMeshDict, snappyHexMeshDict, and surface STL output.

B. Validation

The OpenFOAM export path is runnable in principle, but OpenFOAM was not installed on the host used for this revision. External comparison numbers therefore come from a frozen, dated corpus and are reported as inherited artifacts, not as a fresh local solver run. The corrected comparison convention is documented, but the corpus does not establish external agreement: only 1/16 sonicFoam

rows and 0/14 simpleFoam rows meet the repository's 5% threshold. A fresh OpenFOAM comparison must report residual histories, force-history stability, convergence, boundary conditions, and the post-fix solver version before it can support a validation claim.

The paper therefore claims only that the export path and comparison tooling exist, and that the measurement-integrity audit exposes the current disagreement and its normalization sensitivity. It does not claim a direct validated solver-to-solver result from this revision.

VI. Conclusion

This paper reports the measurement-driven repair of aircraft-like voxel-geometry generation in a proof-of-concept pipeline, evaluated under a declared protocol. The earlier checkpoints failed the repository's own aircraft-specific validity checks at every resolution. Five root causes were isolated by measurement, repaired, and re-evaluated on nine held-out records with an uncalibrated threshold: reconstruction now passes 9/9 and generated samples pass 45/45 under both the shipped four-step sampler and a teacher-DDIM sampler, and the artifacts re-screen as 45/45 through the repository's own CLI. The result is bounded in a way that matters: the 45 draws contain five near-identical voxel arrays, every emitted voxel is already present in the training corpus, and a model-free constant array passes the same screen on every held-out record. The paper therefore demonstrates that the current heuristic gate is satisfiable by a training-hull emitter; it does not demonstrate novel, conditioned, or diverse generation. The held-out latent codes are near-degenerate and the training split holds six records, which is the measured reason. Corpus scale, novelty/diversity gating, and conditioning are the next experiments, not geometric validity alone.

The direct solver-in-loop path is an implementation mechanism, not a validated aerodynamic result. The corrected reference-area audit shows why that distinction matters: a formerly passing comparison row was a normalization coincidence, and the corrected frozen corpus meets the 5% gate on only 1/16 sonicFoam and 0/14 simpleFoam rows. The n=12 condition-response sweep is non-significant and includes documented D3Q27 instability events and same-seed run-to-run variation. These findings discipline the generation result rather than replace it: the generator is now measured against its own screen, and the scorer is still unvalidated.

Most importantly, the repository should not yet be described as a fully AI-driven airplane generator conditioned on flight profile, manufacturing method, or structural requirements. What is demonstrated is narrower and concrete: a checkpoint trained on this host produces voxel geometries that pass every aircraft-specific validity check on held-out records at a declared uncalibrated threshold, together with structured-conditioning plumbing, a direct internal-solver training term with black-box SPSA gradients, and a sequential candidate optimizer that can reuse measured scores after generation. Stronger claims need stronger evidence: larger aircraft corpora with per-condition diversity, normalized conditioning, conditional ablations, stricter structural and aerodynamic checks, and OpenFOAM or comparable PDE validation.

The strongest defensible contribution is therefore the measured validity result together with the measurement discipline around it: the diagnosis and repair of five generation defects, the held-out protocol with an uncalibrated screen, the demonstration that validity alone does not imply generative diversity, the reference-area correction, the quantified reproducibility failure mode, and the power-inadequate condition-response result. The 128^3 cloud protocol is retained as a future experiment, not as a result or an implied promise. Aircraft diversity and conditioning, aerodynamic optimization, structural viability, and superiority remain future work until their explicit gates pass.

References

[1] A. Elham and M. J. L. van Tooren, "Discrete adjoint aerodynamic shape optimization using symbolic analysis with openfemflow," Structural and Multidisciplinary Optimization, vol. 63, pp. 2531–2551, 2021.

[2] E. Andrés-Pérez and C. Paulete-Periáñez, "On the application of surrogate regression models for aerodynamic coefficient prediction," Complex & Intelligent Systems, vol. 7, pp. 1991–2021, 2021.

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," Advances in Neural Information Processing Systems, vol. 33, pp. 6840–6851, 2020.

[4] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in Proceedings of the 38th International Conference on Machine Learning, vol. 139. PMLR, 2021, pp. 8162–8171.

[5] A. Nichol, H. Jun, P. Dhariwal, P. Mishkin, and M. Chen, "Point-e: A system for generating 3d point clouds from complex prompts," arXiv preprint arXiv:2212.08751, 2022.

[6] H. Jun and A. Nichol, "Shap-e: Generating conditional 3d implicit functions," arXiv preprint arXiv:2305.02463, 2023.

[7] J. Wu, O. Sigmund, and J. P. Groen, "Topology optimization of multi-scale structures: a review," Structural and Multidisciplinary Optimization, vol. 63, pp. 1455–1480, 2021.

[8] O. Sigmund, "A 99 line topology optimization code written in matlab," Structural and Multidisciplinary Optimization, vol. 21, pp. 120–127, 2001.

[9] M. P. Bendsøe and O. Sigmund, Topology Optimization: Theory, Methods, and Applications. Springer Science & Business Media, 2013.

[10] I. Sosnovik and I. Oseledets, "Neural networks for topology optimization," Structural and Multidisciplinary Optimization, vol. 60, no. 5, pp. 1387–1401, 2019.

[11] A. Jameson, "Aerodynamic design via control theory," Journal of Scientific Computing, vol. 3, pp. 233–260, 1988.

[12] ——, "Optimum aerodynamic design via boundary control," in 12th Computational Fluid Dynamics Conference, 1995, p. 2432.

[13] A. Forrester, A. Sobester, and A. Keane, Engineering design via surrogate modelling: a practical guide. John Wiley & Sons, 2008.

[14] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," Journal of Computational Physics, vol. 378, pp. 686–707, 2019.

[15] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," arXiv preprint arXiv:2010.08895, 2020.

[16] L. Lu, P. Jin, G. Pang, and G. E. Karniadakis, "Learning nonlinear operators via deeponet based on the universal approximation theorem of operators," Nature Machine Intelligence, vol. 3, no. 3, pp. 218–229, 2021.

[17] Y. Li, Y. Sun, P. Ma, E. Sifakis, T. Du, B. Zhu, and W. Matusik, "Neuralfluid: Neural fluidic system design and control with differentiable simulation," arXiv preprint arXiv:2405.14903, 2024.
[18] R. Vinuesa and S. L. Brunton, "Enhancing computational fluid dynamics with machine learning," Nature Computational Science, vol. 2, no. 6, pp. 358–366, 2022.
[19] Z. Wang, K. Zhang, J. Chu, and H. Ma, "Physics-informed machine learning for aero-structural design optimization," Progress in Aerospace Sciences, vol. 136, p. 100868, 2023.
[20] S. Oh, Y. Jung, S. Kim, I. Lee, and N. Kang, "Deep learning-based generative design for skeleton-optimized structures," Engineering with Computers, vol. 35, pp. 1279–1293, 2019.
[21] F. Steinmetz, S. J. Pfertner, and N. R. Gauger, "Generative design of manufacturable structures using differentiable projection," Computer-Aided Design, vol. 145, p. 103168, 2022.
[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017.
[23] J. Wu, C. Zhang, T. Xue, W. T. Freeman, and J. B. Tenenbaum, "Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling," in Advances in Neural Information Processing Systems, vol. 29, 2016.
[24] S. Luo and W. Hu, "Diffusion probabilistic models for 3d point cloud generation," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 2837–2845.
[25] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, "Occupancy networks: Learning 3d reconstruction in function space," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 4460–4470.
[26] OpenVSP Project, "Openvsp: A parametric aircraft geometry tool," https://github.com/OpenVSP/OpenVSP, 2026, accessed 2026-06-20.
[27] ——, "Openvsp nasa open source agreement license page," https://openvsp.org/license.shtml, 2026, accessed 2026-06-20.
[28] ——, "Vsp airshow," https://airshow.openvsp.org/, 2026, accessed 2026-06-20.
[29] M. Schäfer, S. Turek, F. Durst, E. Krause, and E. Rank, "Benchmark computations of laminar flow around a cylinder," in Flow Simulation with High-Performance Computers II. Springer, 1996, pp. 547–566.
[30] M. Geier, A. Pasquali, and M. Schonherr, "The cascaded lattice boltzmann method," Journal of Computational Physics, vol. 300, pp. 516–542, 2015.